DT-3 Design Decisions Document

1. What pages/routes will your application have?

What it means: Routing is the mapping between the URL in the browser (e.g., /dashboard) and the component that shows up on the screen.

Option A: Simple Flat Routes

Every page is a direct link (e.g., /dashboard, /batches, /errors).

Option B: Nested/Parameterized Routes (Best for this project)

You put the IDs in the URL so users can bookmark a specific view.

Option C: Single Page "State" Routing

The URL never changes; you just swap components based on a variable. (Bad for "Back" button support).

The Plan for your Project:

- /login: Authentication page.
- /dashboard: The high-level overview.
- /batches: The list of all transactions (FR-2.1).
- /batches/🆔 The detail view for one specific batch (FR-2.4).
- /errors: The error analysis and remediation page (FR-3).
- /activity-log: The audit trail (FR-4).

---

2. What reusable components can you identify?

What it means: Don't write the same code twice. If a "Status Badge" looks the same on the Dashboard and the Batch list, make it one component.

Small Components (Atomic):

StatusBadge

Takes a status (e.g., "Failed") and returns a colored pill (Red).

StatCard

A box used on the dashboard to show a single number and a title.

AppButton

A button with loading spinner logic built-in.

Complex Components (Structural):

DataTable

A generic table that handles pagination and sorting logic so you don't have to rewrite it for Batches, Files, and Logs.

SiteSelector

The dropdown that appears in the top navigation on every page.

RefreshTimer

A component that shows "Refreshing in X seconds" and triggers an update.

---

3. How will you manage state?

What it means: "State" is how the app remembers data. If you fetch the Batch list, where do you store it so you don't have to fetch it again if the user clicks "Back"?

Option A: Simple Observables/Services (Good for Mid-size)

You use a Service class to hold the data. Components "subscribe" to it.

Option B: NgRx/Redux (Overkill)

A giant central store. Great for huge teams, but makes simple tasks take 5x longer to code.

Option C: Signals (Modern & Fast)

Available in Angular 16+ or Vue. It’ s the easiest way to make the UI update automatically when data changes.

The Plan for your Project:

Use Services with Signals (or BehaviorSubjects).

Why? It’ s lightweight. You’ ll have a SiteService that remembers the current selectedSiteId. Every other component will "watch" that signal.

---

4. How will you handle the tenant/site selection globally?

What it means: Since FR-5.2 says "All data must be scoped to the selected site," you need a way to ensure that when the user changes the site, every chart and table on the screen updates.

Option A: URL-based (Recommended)

Put the site ID in the URL: app.com/site/123/dashboard. When the user picks a new site, you navigate to a new URL. The code automatically detects the change and re-fetches data.

Option B: Header Component + Global State

A dropdown in the top-bar updates a GlobalState.currentSiteId. All components have a "watcher" that triggers a fetchData() function whenever that ID changes.

Option C: LocalStorage

Save the site ID in the browser memory. (Harder to keep components in sync).

The Plan for your Project:

Option B. A SiteContextService holds the current selection. All API calls look at this service to get the site_id for the request.

---

5. How will you implement auto-refresh on the dashboard?

What it means: FR-1.5 requires the dashboard to refresh (e.g., every 30s) without the user clicking anything.

Option A: setInterval in the Component

When the dashboard opens, start a timer. Every 30 seconds, call loadData(). (Must be careful to stop the timer when the user leaves the page!).

Option B: RxJS timer or interval

A stream that emits every 30s. You link your data fetch to this stream.

Option C: WebSocket (Stretch Goal S-1)

The server "pushes" updates to the frontend. (More complex to build).

The Plan for your Project:

Option B (RxJS Polling). It’ s cleaner than standard JavaScript timers. You create a stream that "pulses" every 30 seconds. On every pulse, you refresh the DashboardSummary and RecentFailures components.

Summary for your DT-3 Document

Problem	Decision	Why?
Routing	Nested Routes	Allows bookmarking and deep-linking to batches.
Components	Atomic Design	Ensures a "Failed" status looks the same across 5 different pages.
State	Signals/Services	Fastest development time with excellent performance.
Tenant Sync	Global Service	Simplifies API calls; components don't need to pass IDs around.
Auto-Refresh	RxJS Polling	Reliable and easy to cancel when the user leaves the dashboard.